

פרויקט בקורס הנדסת תוכנה

BOMBERMAN

מאפיון ועד משחק שרץ באמת

חמישה שלבים · עברית ואנגלית · כל 84 הבדיקות עוברות · Python + Pygame

[שמות חברי הצוות] · [שם הקורס והמרצה] · [תאריך ההצגה]

משחק מבוך קלאסי, שלם ומוכן להצגה

5
שלבים, כל אחד קשה מהקודם

2
שפות: עברית ואנגלית

10
צלילים, ועוד מוזיקת רקע

84
בדיקות אוטומטיות שעוברות

הגרפיקה מצוירת בקוד והצלילים נוצרים בקוד, בלי קובצי מדיה, ולכן המשחק עצמאי לגמרי ורץ על כל מערכת הפעלה

מפוצצים לבנים, מחסלים אויבים, מוצאים את היציאה



פצצות

מתפוצצות אחרי 2.5 שניות. הלהבה נעצרת בקיר והורסת רק את הלבנה הראשונה בכל כיוון



אויבים

האויב הוורוד נודד באקראי. הסגול רודף אחרי השחקן במסלול הקצר ביותר



שיפורים

האות B מוסיפה פצצה, F מגדילה את טווח הפיצוץ, ו-S מאיצה את השחקן



יציאה

מוסתרת מתחת ללבנה, ונפתחת רק אחרי שכל האויבים מתו

צלילים, עברית וחמישה שלבים

צלילים ומוזיקה

אפקט קולי לכל אירוע ומוזיקת רקע. הכול נוצר בקוד, בלי קובצי שמע, ומקש M משתיק

עברית ואנגלית

בוחרים שפה בתפריט. העברית מוצגת מימין לשמאל, ושורת המצב מתהפכת

חמישה שלבים

בשלבים 4 ו-5 יש יותר לבנים, ועד שבעה אויבים סגולים שרודפים אחריך

הבחירה בשפה ובצלילים נשמרת בקובץ settings.json, ובהפעלה הבאה המשחק נפתח באותו מצב

יש 29 דרישות, ולכל אחת מהן יש דרך לבדוק אותה

דרישות חובה

14

טעינת 5 שלבים ואימות

תנועה, פצצות ופיצוצים

אויבים ומגע

חיים, עצירה ומעבר שלב

שיאים, תפריט ושורת מצב

צלילים ובחירת שפה

דרישות רצויות

3

תגובת שרשרת

שיפורים

שעון לכל שלב

דרישות לא-פונקציונליות

12

קצב של 60 פריימים לשנייה

כיסוי בדיקות של 90% לפחות

שלב חדש הוא קובץ טקסט אחד

שפה חדשה בלי שינוי לוגיקה

הלוגיקה נפרדת מהתצוגה ונבדקת בלי חלון

שכבת התצוגה
השכבה היחידה שתלויה בספריית הגרפיקה

- **app.py** — לולאה, קלט ומסכים
- **renderer.py** — ציור המסך
- **sound.py** — צלילים ומוזיקה

קוראת ל-update ומציירת את המצב

שכבת הלוגיקה
קוד נקי, בלי גרפיקה

- **game.py** — חוקים ומצבים
- **ai.py** — נדידה ומרדף
- **entities.py** — שחקן, אויב, פצצה
- **grid.py** — מפת המשבצות
- **level.py** — טעינת 5 שלבים

שירותים משותפים
גם הם בלי גרפיקה, ונבדקים אוטומטית

- **i18n.py** — תרגומים
- **text.py** — עברית מימין לשמאל
- **settings.py** — שפה וצלילים
- **highscores.py** — טבלת שיאים

מכונת המצבים

שבעה מצבים, והמשחק תמיד חוזר לתפריט



PAUSED

לחיצה על P או Esc עוצרת את המשחק. השעון, הפצצות והאויבים קופאים

PLAYER_DIED

אם נשארו חיים, השלב מתחיל מחדש. אחרת המשחק עובר למסך הסיום

GAME_OVER

לחיצה על Enter מובילה לטבלת השיאים, ומשם חוזרים לתפריט

שני רעיונות שעושים את המשחק חכם

האויב הסגול רודף בעזרת חיפוש לרוחב

חיפוש לרוחב (BFS) מוצא את הצעד הראשון במסלול הקצר ביותר אל השחקן, ועוקף קירות ופצצות

```
queue = deque([(start, None)])
while queue:
    pos, first = queue.popleft()
    for d in Direction:
        nxt = step(pos, d)
        if nxt == goal: return first or d
        if walkable(nxt) and nxt not in seen:
            seen.add(nxt)
            queue.append((nxt, first or d))
```

תגובת שרשרת עם תור

כל פצצה שהלהבה פוגעת בה נכנסת לתור. כל פצצה מתפוצצת פעם אחת בלבד

```
queue = list(due_bombs)
while queue:
    bomb = queue.pop()
    if bomb in done: continue
    done.append(bomb)
    for tile in flame_path(bomb):
        other = bomb_at(tile)
        if other and other not in done:
            queue.append(other)
```

כל 84 הבדיקות עוברות בפחות משנייה

קובץ	מה נבדק	בדיקות
test_level.py	טעינת 5 שלבים, אימות ומפה	17
test_player.py	תנועה, חסימות ושיפורים	9
test_bombs.py	פצצות, להבות ושרשרת	9
test_enemies.py	נדידה, מרדף BFS, מגע ופגיעה בלהבה	11
test_game_flow.py	חיים, שעון, עצירה ומעבר שלב	10
test_highscores.py	מיון, שמירה וקובץ פגום	6
test_localization.py	תרגומים, עברית והגדרות	15
test_sound.py	סינתוז צלילים ויצירת קובצי שמע	7

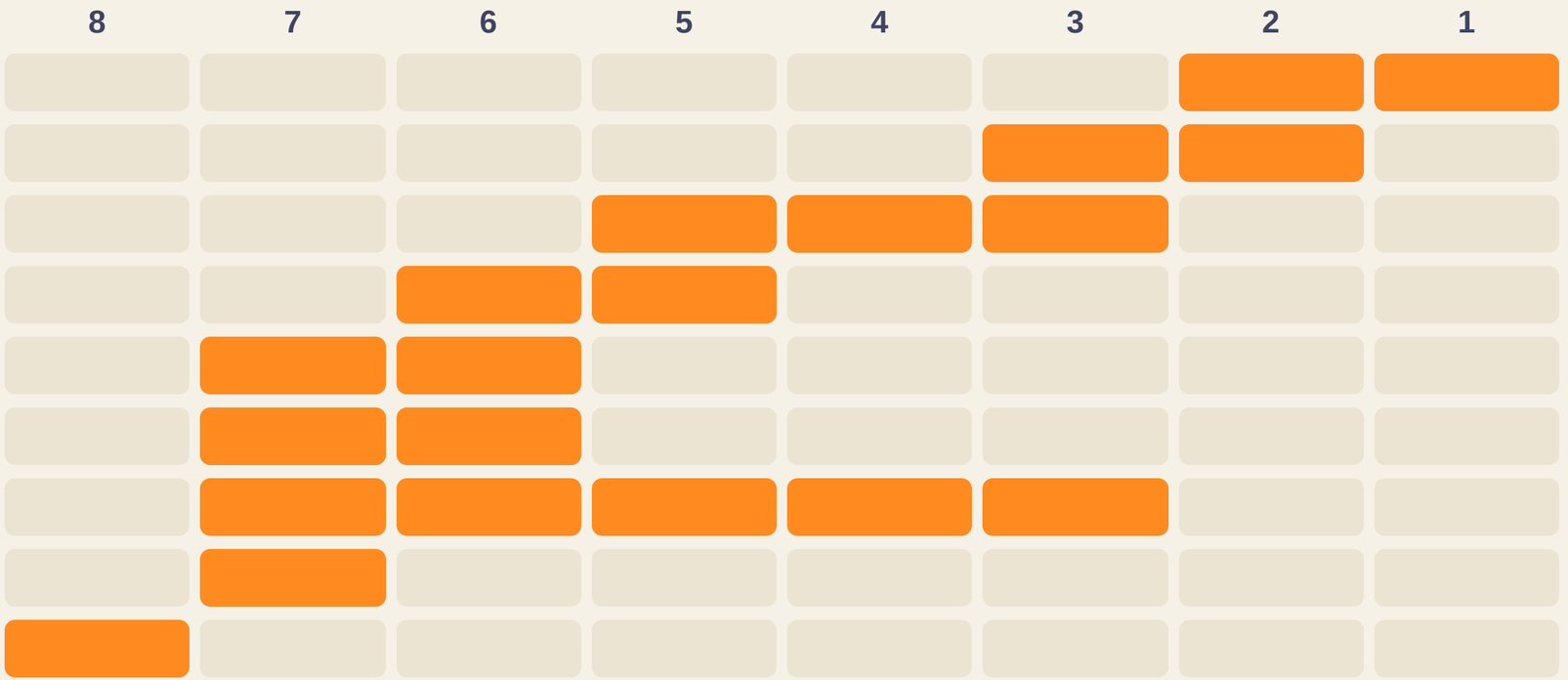
95%

בערך זה הכיסוי של קוד הלוגיקה

כל דרישה פונקציונלית מקושרת לבדיקה במטריצת העקיבות שבמסמך האפיון

```
python -m pytest -v
```

שמונה שבועות, ובדיקות לאורך כל הדרך



חלוקת תפקידים

כל חבר צוות אחראי על שכבה ועל הבדיקות שלה

[שם]

ניהול ואפיון

מסמך האפיון, לוח הזמנים, מעקב דרישות והמצגת

`README`
`config.py`

[שם]

לוגיקת המשחק

מודל המשחק, מכונת המצבים, פצצות והתנגשויות

`game.py`
`entities.py`
`grid.py`

[שם]

אויבים ושלבים

בינה מלאכותית של אויבים, פורמט שלבים ומחולל השלבים

`ai.py`
`level.py`
`levels/`

[שם]

ממשק ואיכות

לולאת המשחק, ציור, תפריטים, צלילים, שפות ותשתית הבדיקות

`app.py`
`renderer.py`
`sound.py`
`i18n.py`
`tests/`

הדגמה

שלוש פקודות, ומשחקים

```
python -m pip install -r requirements.txt
python main.py
python -m pytest -v
```

← ↑ → ↓	Space	P / Esc	M	Enter
תנועה	הנחת פצצה	עצירה	השתקת צלילים	אישור

תכנון טוב בהתחלה חסך לנו זמן בסוף

01

הפרדה בין לוגיקה לתצוגה הפכה את הבדיקות לפשוטות ומהירות

02

דרישות עם קריטריון מדיד סגרו ויכוחים לפני שהתחלנו לכתוב קוד

03

מטריצת העקיבות הראתה מיד אילו דרישות עוד לא נבדקו

שאלות?